

LIVRET D'EXCELLENCE TOUT-EN-UN

Sciences de l'Informatique (Baccalauréat Tunisien)

BASES DE DONNÉES

Chapitre 6 : Manipulation d'une Base de Données

Le Langage de Manipulation de Données (SQL - LMD)

Conforme au Programme Officiel de la 4ème Année Secondaire

Synthèse de Cours • 40 Flashcards d'Apprentissage Actif • QCM & Évaluations du Bac

Contents

1	Fiche 1 : Le Cours Complet (SQL - LMD)	2
1.1	Introduction au Langage de Manipulation de Données (LMD)	2
1.2	Insertion de données (INSERT INTO)	2
1.2.1	Syntaxe Générale avec ciblage des colonnes	2
1.2.2	Syntaxe Globale (sans spécification des colonnes)	2
1.3	Mise à jour de données (UPDATE)	3
1.3.1	Syntaxe Générale	3
1.3.2	Avertissement Critique (Le piège du WHERE)	3
1.4	Suppression de données (DELETE)	3
1.4.1	Syntaxe Générale	3
1.4.2	Attention au comportement en cascade et au WHERE	3
1.5	Interrogation des données (SELECT)	5
1.5.1	Syntaxe Générale Universelle	5
1.5.2	Les Clauses Clés Décortiquées	5
1.5.3	Les Opérateurs de Filtrage (dans la clause WHERE)	5
1.5.4	Les Fonctions d'Agrégation (Calculs Statistiques)	5
1.5.5	La Jointure de Tables (WHERE vs JOIN)	5
1.6	Modélisation Graphique TikZ du Filtrage Relationnel	7
2	Fiche 2 : Le Recueil de Questions-Réponses (Active Recall)	8
3	Fiche 3 : Le Quiz d'Évaluation & Fiches Mémo	12
3.1	Quiz d'Évaluation d'Excellence (10 QCM d'Examen)	12
3.2	Le Grand Mémento de la Syntaxe SQL - LMD	15
3.3	Fiche Mémo Visuelle : Les 5 pièges classiques du SQL au Bac	15

Fiche 1 : Le Cours Complet (SQL - LMD)

1.1 Introduction au Langage de Manipulation de Données (LMD)

Le **LMD (Language de Manipulation de Données)** est le sous-ensemble de SQL destiné à manipuler les données au sein d'une structure de base de données déjà créée par le LDD. Contrairement au LDD qui agit sur la structure (tables, colonnes), le LMD agit sur les **uplets (lignes)** des tables. Il permet d'effectuer quatre opérations fondamentales résumées par l'acronyme **CRUD** :

- **C (Create)** : Insertion de nouveaux uplets → instruction **INSERT**.
- **R (Read)** : Lecture et sélection d'uplets → instruction **SELECT**.
- **U (Update)** : Modification d'uplets existants → instruction **UPDATE**.
- **D (Delete)** : Suppression d'uplets → instruction **DELETE**.

1.2 Insertion de données (INSERT INTO)

La clause **INSERT INTO** permet d'insérer un ou plusieurs nouveaux enregistrements (uplets) dans une table.

1.2.1 Syntaxe Générale avec ciblage des colonnes

Cette approche est fortement recommandée au Baccalauréat car elle est indépendante de l'ordre physique des colonnes dans la table. Elle permet également d'omettre les colonnes qui acceptent les valeurs nulles ou bénéficient d'une valeur par défaut (**DEFAULT**).

Syntaxe d'insertion ciblée

```
INSERT INTO nomitable(colonne1,colonne2,...,colonneN)VALUES(valeur1,valeur2,...,valeurN);
```

1.2.2 Syntaxe Globale (sans spécification des colonnes)

Cette syntaxe impose de fournir une valeur pour chaque colonne de la table, dans l'ordre exact défini lors de la création de la table.

Syntaxe d'insertion globale

```
INSERT INTO nomitableVALUES(valeur1,valeur2,...,valeurN);
```

Exemples Pratiques d'Insertion

Soit la table **ELEVE** (**IdEleve**, **Nom**, **Prenom**, **DateNaissance**, **Moyenne**):

- **Exemple 1 (Ciblé)** : Insertion d'un élève en omettant la moyenne (qui prendra la valeur par défaut ou **NULL**):

```
INSERT INTO ELEVE (IdEleve, Nom, Prenom, DateNaissance)
VALUES (105, 'Trabelsi', 'Mohamed', '2008-05-14');
```

- **Exemple 2 (Global)** : Insertion complète :

```
INSERT INTO ELEVE
VALUES (106, 'Ben Ali', 'Sarrah', '2008-11-23', 15.75);
```

1.3 Mise à jour de données (UPDATE)

L'instruction UPDATE permet de modifier les valeurs des attributs d'un ou de plusieurs enregistrements existants dans une table.

1.3.1 Syntaxe Générale

Syntaxe de modification

```
UPDATE nom_table SET colonne1 = nouvelle_valeur1, colonne2 = nouvelle_valeur2, ... [WHERE condition];
```

1.3.2 Avertissement Critique (Le piège du WHERE)

Si la clause WHERE est omise, la mise à jour s'appliquera à **toutes les lignes** de la table sans exception ! C'est l'une des erreurs les plus pénalisantes du Baccalauréat pratique et théorique.

Exemples Pratiques de Mise à jour

- **Exemple 1** : Augmenter la moyenne de l'élève ayant l'ID 105 de 1 point :

```
UPDATE ELEVE
SET Moyenne = Moyenne + 1
WHERE IdEleve = 105;
```

- **Exemple 2** : Réinitialiser la moyenne de tous les élèves nés après '2008-01-01' à 10 :

```
UPDATE ELEVE
SET Moyenne = 10
WHERE DateNaissance > '2008-01-01';
```

1.4 Suppression de données (DELETE)

L'instruction DELETE FROM permet de supprimer définitivement un ou plusieurs enregistrements d'une table.

1.4.1 Syntaxe Générale

Syntaxe de suppression d'enregistrements

```
DELETE FROM nom_table [WHERE condition];
```

1.4.2 Attention au comportement en cascade et au WHERE

- Comme pour l'instruction UPDATE, si la clause WHERE est omise, **toutes les lignes de la table seront définitivement supprimées**, bien que la structure de la table reste intacte.
- **Différence cruciale entre DELETE et DROP TABLE** : DELETE vide la table mais conserve sa structure (le contenant reste, le contenu part). DROP TABLE détruit la table et sa structure de la base de données (le contenant et le contenu partent).

Exemples Pratiques de Suppression

- **Exemple 1** : Supprimer l'élève ayant l'ID 106 :

```
DELETE FROM ELEVE
```

```
WHERE IdEleve = 106;
```

- **Exemple 2** : Supprimer tous les élèves ayant une moyenne strictement inférieure à 9.00 :

```
DELETE FROM ELEVE  
WHERE Moyenne < 9.00;
```

1.5 Interrogation des données (SELECT)

L'instruction **SELECT** est de loin la plus riche et la plus utilisée en SQL. Elle permet d'extraire et de formater des informations en mémoire de travail à partir d'une ou plusieurs tables.

1.5.1 Syntaxe Générale Universelle

Structure globale d'une requête d'interrogation

```
SELECT [DISTINCT] liste_colonnes ou expressions FROM liste_tables [WHERE conditions de filtrage et jointure] [GROUP BY colonnes]
```

1.5.2 Les Clauses Clés Décortiquées

1. **SELECT** : Spécifie les colonnes à afficher. L'utilisation du caractère ***** permet d'afficher la totalité des colonnes. La clause optionnelle **DISTINCT** élimine les doublons de l'affichage final.
2. **FROM** : Liste la ou les tables sources contenant les données.
3. **WHERE** : Définit les critères de sélection (restriction) ou les conditions de jointure de tables.
4. **GROUP BY** : Permet de regrouper les enregistrements ayant des valeurs identiques pour les colonnes spécifiées afin de calculer des statistiques (sommés, moyennes, comptages).
5. **HAVING** : Filtre les groupes produits par **GROUP BY**. *Règle d'or : On utilise HAVING uniquement pour poser des conditions sur des fonctions d'agrégation (ex: HAVING COUNT(*) > 5).*
6. **ORDER BY** : Trie les lignes du résultat final de manière croissante (**ASC**, par défaut) ou décroissante (**DESC**).

1.5.3 Les Opérateurs de Filtrage (dans la clause WHERE)

Le Baccalauréat tunisien requiert la maîtrise d'opérateurs de comparaison et d'évaluation précis :

- **Comparaison classique** : =, <> (différent de), <, >, <=, >=.
- **Opérateurs logiques** : AND, OR, NOT.
- **Opérateur d'intervalle** : BETWEEN val1 AND val2 (les bornes sont incluses).
- **Opérateur d'ensemble** : IN (val1, val2, ...) (renvoie vrai si la valeur est dans la liste).
- **Recherche de motifs (LIKE)** : Permet la recherche floue à l'aide de caractères génériques :
 - % : Remplace une chaîne de 0, 1 ou plusieurs caractères quelconques.
 - _ (souligné) : Remplace exactement un seul caractère quelconque.
- **Valeurs Nulles** : IS NULL et IS NOT NULL (indispensable car = NULL ne fonctionne pas).

1.5.4 Les Fonctions d'Agrégation (Calculs Statistiques)

Ces fonctions s'appliquent sur un ensemble de lignes pour retourner une valeur unique :

- **COUNT(colonne)** ou **COUNT(*)** : Compte le nombre d'enregistrements.
- **SUM(colonne)** : Calcule la somme des valeurs d'une colonne numérique.
- **AVG(colonne)** : Calcule la moyenne des valeurs d'une colonne numérique.
- **MAX(colonne)** : Retourne la valeur maximale.
- **MIN(colonne)** : Retourne la valeur minimale.

1.5.5 La Jointure de Tables (WHERE vs JOIN)

Lorsque les informations recherchées résident dans plusieurs tables, il est nécessaire de les relier via un attribut commun (généralement la paire Clé Primaire - Clé Étrangère). C'est la **jointure**.

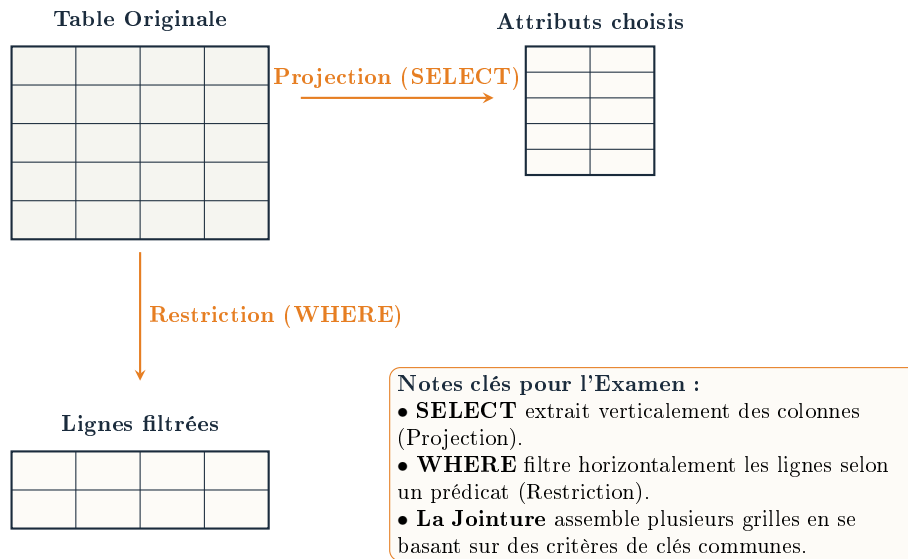
Jointure classique via la clause WHERE

```
SELECT ELEVE.Nom, CLASSE.NomClasse FROM ELEVE, CLASSE WHERE ELEVE.IdClasse = CLASSE.IdClasse; -  
Condition de jointure obligatoire !
```

Remarque : Si une condition de jointure est omise, le SGBD effectue un produit cartésien (chaque ligne de la première table est associée à toutes les lignes de la deuxième table), ce qui est une faute majeure.

1.6 Modélisation Graphique TikZ du Filtrage Relationnel

Voici une représentation visuelle de la différence fondamentale entre les opérations d'extraction algébriques de base en bases de données : la **Projection** (sélection de colonnes) et la **Restriction** (sélection de lignes), ainsi que leur combinaison.



Fiche 2 : Le Recueil de Questions-Réponses (Active Recall)

Ce recueil comprend 40 questions d'excellence conçues comme un système de flashcards fermées. Utilisez-les pour réviser activement les détails indispensables du SQL LMD.

Question 1 : Quelle est la fonction principale du Langage de Manipulation de Données (LMD) ?

Le LMD permet de manipuler les enregistrements d'une base de données existante. Il offre les commandes pour consulter (SELECT), insérer (INSERT), modifier (UPDATE) et supprimer (DELETE) les lignes des tables.

Question 2 : Quelle commande permet d'ajouter une nouvelle ligne dans une table ?

C'est la commande INSERT INTO suivie du nom de la table, éventuellement de la liste des colonnes, et de la clause VALUES contenant les données à insérer.

Question 3 : Pourquoi est-il fortement recommandé de spécifier la liste des colonnes lors d'un INSERT ?

Cela rend la requête plus robuste. Si la structure physique de la table change dans le futur (ajout ou suppression d'une colonne facultative), la requête d'insertion continuera de fonctionner correctement. De plus, cela évite de devoir se rappeler de l'ordre exact des colonnes lors de la création de la table.

Question 4 : Que se passe-t-il si l'on tente d'insérer une ligne en omettant une colonne déclarée NOT NULL sans valeur ?

Le SGBD lève une erreur de contrainte d'intégrité de domaine et refuse catégoriquement l'insertion de l'enregistrement.

Question 5 : Comment écrit-on des valeurs textuelles ou de dates en SQL ?

Toutes les valeurs de type chaîne de caractères (CHAR, VARCHAR) et de type temporel (DATE) doivent obligatoirement être entourées de guillemets simples (apostrophes, ex: 'Tunis' ou '2008-09-05'). Les nombres ne doivent pas porter de guillemets.

Question 6 : Quelle instruction permet d'éditer des valeurs existantes dans une table ?

C'est l'instruction UPDATE suivie du nom de la table, de la clause SET spécifiant les attributs et leurs nouvelles valeurs, et de la clause WHERE pour limiter la modification.

Question 7 : Quel est le danger majeur de l'omission de la clause WHERE dans un UPDATE ?

La modification s'applique sans avertissement à **tous les enregistrements de la table**. Par exemple, écrire UPDATE ELEVE SET Moyenne = 20; donne un 20/20 à l'intégralité des élèves de la base de données.

Question 8 : Quelle instruction permet de vider une table de ses données sans supprimer sa structure ?

C'est l'instruction DELETE FROM suivie du nom de la table, sans clause WHERE : DELETE FROM nom_table; La structure de la table (déclarations de colonnes, contraintes) reste représentée dans le dictionnaire de la base.

Question 9 : En quoi DELETE FROM ELEVE; diffère-t-il de DROP TABLE ELEVE; ?

DELETE FROM ELEVE; supprime uniquement toutes les lignes (le contenu) mais conserve la table ELEVE qui reste utilisable. DROP TABLE ELEVE; supprime la table elle-même de la base (contenant et contenu) ; la table n'existe plus du tout après cette commande.

Question 10 : À quoi sert l'opérateur DISTINCT placé après la clause SELECT ?

L'opérateur DISTINCT élimine de l'affichage final toutes les lignes en double. Il permet d'obtenir des résultats uniques (par exemple, lister les villes des élèves sans répétition).

Question 11 : Quel symbole permet de sélectionner tous les attributs d'une table dans une requête SELECT ?

C'est le caractère étoile (*). Exemple : SELECT * FROM ELEVE;.

Question 12 : Quel opérateur permet d'effectuer un filtrage basé sur un intervalle de valeurs ?

C'est l'opérateur BETWEEN *val_{min} AND val_{max}. Les valeurs bornes*

Question 13 : Quel opérateur permet de vérifier si une valeur appartient à une liste ou un ensemble précis ?

C'est l'opérateur IN. Exemple : WHERE Ville IN ('Tunis', 'Sousse', 'Sfax').

Question 14 : À quoi sert l'opérateur LIKE et quels sont ses deux métacaractères en SQL ?

LIKE permet d'effectuer des recherches de motifs flous (textuels). Ses deux métacaractères sont : 1. Le pourcentage (%) qui remplace n'importe quelle chaîne de 0, 1 ou plusieurs caractères. 2. Le souligné (_) qui remplace exactement un seul caractère quelconque.

Question 15 : Quelle condition permet de sélectionner les lignes dont le nom commence par 'A' ?

La clause est : WHERE Nom LIKE 'A%'.

Question 16 : Quelle condition permet de sélectionner les noms de 5 lettres dont la 3ème lettre est 'R' ?

La clause est : WHERE Nom LIKE '__R__'.

Question 17 : Pourquoi la clause WHERE Moyenne

En SQL, la valeur spéciale NULL représente l'absence de données (valeur inconnue). Elle ne peut pas être comparée avec l'opérateur d'égalité standard =. Il faut obligatoirement utiliser l'opérateur IS NULL (ou IS NOT NULL).

Question 18 : Quels sont les opérateurs logiques permettant de combiner plusieurs critères dans un WHERE ?

Ce sont les opérateurs logiques standards AND (et), OR (ou), et NOT (non).

Question 19 : Comment ordonne-t-on le résultat d'une requête d'interrogation ?

On ajoute la clause ORDER BY en fin de requête, suivie des noms des colonnes de tri, puis du mot clé ASC (croissant, par défaut) ou DESC (décroissant).

Question 20 : Peut-on trier un résultat sur plusieurs colonnes en même temps ?

Oui, en séparant les colonnes de tri par des virgules dans la clause ORDER BY. Exemple : ORDER BY Ville ASC, Moyenne DESC;. Le SGBD triera d'abord par ville, et pour les villes identiques, il triera par moyenne décroissante.

Question 21 : Quelles sont les 5 fonctions d'agrégation standards en SQL ?

Ce sont : COUNT (comptage), SUM (somme), AVG (moyenne), MIN (minimum), et MAX (maximum).

Question 22 : Quelle est la différence entre COUNT(*) et COUNT(Moyenne) ?

COUNT(*) compte le nombre total de lignes renvoyées par la requête, y compris celles contenant des valeurs nulles. COUNT(Moyenne) compte uniquement les lignes où l'attribut Moyenne possède une valeur non nulle (NOT NULL).

Complexité : Les deux opérations s'effectuent en une seule passe sur les données.

Question 23 : Quand est-il obligatoire d'utiliser la clause GROUP BY ?

La clause GROUP BY est obligatoire dès qu'une requête de sélection contient à la fois un attribut « classique » (non agrégé) et une fonction d'agrégation dans sa clause SELECT. Tout attribut présent dans SELECT qui n'est pas enveloppé dans une fonction d'agrégation doit obligatoirement figurer dans GROUP BY.

Question 24 : Quel est le rôle de la clause HAVING ?

HAVING permet de filtrer les groupes d'enregistrements créés par la clause GROUP BY. Elle agit comme un WHERE, mais s'applique exclusivement aux résultats des fonctions d'agrégation.

Question 25 : Pourquoi ne peut-on pas écrire WHERE AVG(Moyenne) >

La clause WHERE filtre les lignes **avant** que les groupes ne soient formés et que les fonctions d'agrégation ne soient calculées. Pour poser une condition sur une moyenne de groupe, il faut obligatoirement utiliser la clause HAVING après le GROUP BY : HAVING AVG(Moyenne) >= 10.

Question 26 : Dans quel ordre logique le SGBD traite-t-il les différentes clauses d'un SELECT ?

L'ordre d'exécution interne du SGBD est le suivant : 1. FROM (sélection des tables sources) 2. WHERE (filtrage des lignes individuelles) 3. GROUP BY (regroupement des lignes restantes) 4. HAVING (filtrage des groupes créés) 5. SELECT (projection et calcul des expressions) 6. DISTINCT (élimination des doublons d'affichage) 7. ORDER BY (tri du résultat final)

Question 27 : Qu'est-ce qu'une jointure ?

Une jointure est une opération combinant les colonnes de plusieurs tables en se basant sur une valeur partagée commune (généralement une relation Clé Primaire de la table parente = Clé Étrangère de la table fille) pour regrouper des données logiquement connexes.

Question 28 : Que se passe-t-il si l'on effectue une requête SELECT sur deux tables sans spécifier de condition

Le SGBD effectue un **produit cartésien**. Il associe chaque ligne de la première table à chaque ligne de la seconde table, produisant un nombre de lignes égal au produit des cardinalités des deux tables (ex: 10 lignes × 50 lignes = 500 lignes), la plupart étant fausses ou incohérentes.

Question 29 : Comment écrit-on une jointure classique dans la clause WHERE ?

On liste les tables dans le FROM et on ajoute la condition d'égalité des clés dans le WHERE. Exemple : FROM ELEVE, CLASSE WHERE ELEVE.IdClasse = CLASSE.IdClasse;

Question 30 : À quoi servent les alias de tables en SQL et comment les déclare-t-on ?

Les alias permettent de renommer temporairement une table dans une requête pour la rendre plus courte et lisible. On les déclare dans le FROM en plaçant l'alias juste après le nom de la table. Exemple : FROM ELEVE E, INSCRIPTION I WHERE E.IdEleve = I.IdEleve;

Question 31 : Quand l'utilisation des préfixes de tables (ex: ELEVE.Nom) est-elle obligatoire ?

Elle est obligatoire pour lever toute ambiguïté lorsque deux tables distinctes impliquées dans la même requête possèdent un attribut ayant exactement le même nom (ex: IdClasse présent dans ELEVE et dans CLASSE).

Question 32 : Comment renomme-t-on l'en-tête d'une colonne dans l'affichage final d'une requête ?

On utilise l'alias de colonne avec le mot clé AS. Exemple : `SELECT Nom, Moyenne * 5 AS NoteSurCent FROM ELEVE;`

Question 33 : Quelle fonction permet d'obtenir l'année courante ou la date système en SQL ?

Selon le SGBD, on utilise des fonctions comme `CURRENT_DATE`, `NOW()` ou `DATE()`. *Au Baccalauréat tunisien, l'écriture de la date est obligatoire, l'écriture de l'heure n'est pas obligatoire.*

Question 34 : L'expression WHERE Ville

Non, c'est une erreur syntaxique fréquente. Chaque condition logique de part et d'autre de l'opérateur OR doit être complète et autonome. La syntaxe correcte est : `WHERE Ville = 'Tunis' OR Ville = 'Sfax'`. Alternativement, on peut écrire : `WHERE Ville IN ('Tunis', 'Sfax')`.

Question 35 : Peut-on insérer des expressions arithmétiques dans la clause SELECT ?

Oui, on peut effectuer des calculs directement sur les attributs numériques. Exemple : `SELECT Nom, (NoteOral + NoteEcrite * 2) / 3 AS MoyenneGenerale FROM ELEVE;`

Question 36 : Est-il possible de faire un GROUP BY sur plusieurs colonnes ?

Oui, le SGBD créera un groupe pour chaque combinaison unique de valeurs de ces colonnes. Exemple : `GROUP BY Option, Sexe;` permet d'obtenir des agrégats (sommes, moyennes) pour chaque binôme d'Option et de Sexe.

Question 37 : Quel opérateur permet d'exclure les éléments d'une liste ?

C'est l'opérateur de négation appliqué à l'ensemble : NOT IN. Exemple : `WHERE Ville NOT IN ('Tunis', 'Sousse')`.

Question 38 : Quel est le rôle d'une contrainte d'intégrité référentielle lors de l'exécution d'un DELETE ?

Si un enregistrement d'une table parente (ex: une classe dans CLASSE) est référencé par des enregistrements d'une table enfant (ex: des élèves dans ELEVE), la suppression de la classe sera bloquée par le SGBD pour préserver l'intégrité, sauf si l'option ON DELETE CASCADE a été configurée à la création.

Question 39 : Comment filtre-t-on les élèves dont le prénom commence par la lettre 'M' et se termine par la lettre 'D' ?

On combine le motif avec le filtre LIKE : `WHERE Prenom LIKE 'M%D'`.

Question 40 : Quelle est la différence majeure d'application entre la clause WHERE et la clause HAVING ?

WHERE filtre les lignes d'une table **individuellement** avant tout regroupement. HAVING filtre des **groupes d'enregistrements** créés par la clause GROUP BY sur la base de critères calculés par des fonctions d'agrégation.

Fiche 3 : Le Quiz d'Évaluation & Fiches

Mémo

3.1 Quiz d'Évaluation d'Excellence (10 QCM d'Examen)

Ce quiz contient 10 questions représentatives des épreuves écrites du Baccalauréat tunisien. Testez vos connaissances et étudiez les corrections d'une rigueur absolue.

QCM 1 : Insertion partielle

Laquelle de ces requêtes est correcte pour insérer un élève ayant les attributs Id (101) et Nom ('Ali') dans la table ELEVE (Id, Nom, Prenom, Note) sachant que Prenom et Note acceptent les valeurs nulles ?

- A) INSERT INTO ELEVE VALUES (101, 'Ali');
- B) INSERT INTO ELEVE (Id, Nom) VALUES (101, 'Ali');
- C) INSERT INTO ELEVE SET Id=101, Nom='Ali';

Correction détaillée :

La réponse correcte est **B**. L'option **A** échoue car l'insertion globale sans spécifier de colonnes impose de fournir des valeurs pour les 4 attributs de la table. L'option **C** utilise une syntaxe propre à MySQL mais non conforme au standard ANSI SQL exigé au Baccalauréat tunisien.

QCM 2 : Le comportement de UPDATE

Soit l'instruction : UPDATE PRODUIT SET Prix = Prix * 0.9 WHERE Categorie = 'Loisir';
Quel est l'impact réel de cette instruction sur la table PRODUIT ?

- A) Elle applique une remise de 10% à tous les produits.
- B) Elle applique une remise de 10% uniquement aux produits de la catégorie 'Loisir'.
- C) Elle remplace le prix de tous les produits par 0.9.

Correction détaillée :

La réponse correcte est **B**. L'instruction applique un coefficient multiplicateur de 0.9 (soit une baisse de 10%) uniquement aux lignes satisfaisant le critère restrictif de la clause WHERE (Categorie = 'Loisir').

QCM 3 : Recherche floue (LIKE)

Quelle clause sélectionne les produits dont le code se termine exactement par le chiffre 9 et possède une longueur totale d'au moins 3 caractères ?

- A) WHERE Code LIKE '%9';
- B) WHERE Code LIKE '__9';
- C) WHERE Code LIKE '___%9';

Correction détaillée :

La réponse correcte est **C**. L'option **A** sélectionne les codes se terminant par 9 mais sans contrainte de

longueur (peut faire 1 ou 2 caractères). L'option **B** impose une taille fixe de **strictement 3 caractères**. L'option **C** exige deux caractères quelconques au début (__) suivis d'un nombre indéterminé de caractères (%) pour finir par un 9, assurant une longueur minimale de 3 caractères.

QCM 4 : Utilisation du GROUP BY

Soit la requête : `SELECT Ville, COUNT(*) FROM ELEVE;`

Cette requête produit une erreur de compilation dans le SGBD. Comment la corriger ?

- A) En remplaçant `COUNT(*)` par `COUNT(Ville)`.
- B) En ajoutant la clause `GROUP BY Ville` à la fin.
- C) En remplaçant `SELECT` par `SELECT DISTINCT`.

Correction détaillée :

La réponse correcte est **B**. Mélanger un attribut individuel (`Ville`) et une fonction d'agrégation (`COUNT`) sans clause de regroupement est interdit. Il faut obligatoirement regrouper les enregistrements par `Ville` pour que le SGBD calcule le nombre d'élèves par ville.

QCM 5 : Restriction de groupe

Quelle clause permet de filtrer uniquement les villes comptant strictement plus de 150 élèves inscrits ?

- A) `WHERE COUNT(*) > 150`
- B) `HAVING COUNT(*) > 150` (après un `GROUP BY Ville`)
- C) `WHERE Ville > 150`

Correction détaillée :

La réponse correcte est **B**. L'option **A** est syntaxiquement invalide car les fonctions d'agrégation comme `COUNT` ne peuvent pas être évaluées dans une clause `WHERE` qui filtre ligne par ligne. Seul `HAVING` permet de filtrer les résultats agrégés d'un groupe.

QCM 6 : Valeurs manquantes

Laquelle de ces conditions extrait correctement les lignes pour lesquelles la note finale n'a pas encore été saisie ?

- A) `WHERE Note = NULL`
- B) `WHERE Note IS NULL`
- C) `WHERE Note = ''`

Correction détaillée :

La réponse correcte est **B**. En SQL, la valeur d'absence d'information `NULL` n'est pas équivalente à une chaîne vide `''` (qui est une valeur connue de longueur 0). On l'évalue obligatoirement via l'opérateur d'état `IS NULL`.

QCM 7 : Opérateur d'appartenance

La requête : `SELECT * FROM ELEVE WHERE IdClasse IN (1, 2, 3);` est équivalente à :

- A) `SELECT * FROM ELEVE WHERE IdClasse = 1 AND IdClasse = 2 AND IdClasse = 3;`
- B) `SELECT * FROM ELEVE WHERE IdClasse = 1 OR IdClasse = 2 OR IdClasse = 3;`
- C) `SELECT * FROM ELEVE WHERE IdClasse BETWEEN 1 AND 3;`

Correction détaillée :

La réponse correcte est **B** (et par extension **C** dans ce cas particulier d'entiers consécutifs). L'opérateur `IN` réalise une disjonction logique (`OR`). L'option **A** est fausse car une variable ne peut pas avoir simultanément trois valeurs distinctes (le résultat serait toujours vide).

QCM 8 : Tri décroissant

Quelle clause permet de trier les résultats du plus récent au plus ancien par date de naissance ?

- A) ORDER BY DateNaissance ASC
- B) ORDER BY DateNaissance DESC
- C) SORT BY DateNaissance BACKWARD

Correction détaillée :

La réponse correcte est **B**. Les dates les plus récentes ayant des valeurs numériques supérieures dans la représentation interne, un tri du plus récent au plus ancien correspond à un tri décroissant (DESC). L'instruction SORT BY n'existe pas en SQL standard.

QCM 9 : Effet du DELETE sans WHERE

Que provoque l'exécution de la requête : DELETE FROM CLASSE; ?

- A) La suppression de la table CLASSE de l'ordinateur.
- B) La suppression de toutes les lignes de la table CLASSE, rendant la table vide.
- C) Une erreur de syntaxe car la clause WHERE est obligatoire.

Correction détaillée :

La réponse correcte est **B**. Bien que très dangereux, un DELETE sans clause de filtrage WHERE est parfaitement valide pour le SGBD. Il vide intégralement la table de toutes ses données tout en conservant sa structure physique.

QCM 10 : Jointures de tables

Soit deux tables : CLIENT (IdClient, Nom) et COMMANDE (IdCmd, DateCmd, IdClient).
Comment afficher le nom du client associé à chaque commande ?

- A) SELECT Nom FROM CLIENT, COMMANDE;
- B) SELECT Nom FROM CLIENT, COMMANDE WHERE CLIENT.IdClient = COMMANDE.IdClient;
- C) SELECT Nom FROM CLIENT JOIN COMMANDE;

Correction détaillée :

La réponse correcte est **B**. L'option **A** réalise un produit cartésien associant de faux noms aux commandes. L'option **C** utilise une syntaxe de jointure interne incomplète car elle ne mentionne pas la clause de liaison ON CLIENT.IdClient = COMMANDE.IdClient.

3.2 Le Grand Mémento de la Syntaxe SQL - LMD

Ce tableau récapitulatif universel doit être mémorisé pour maîtriser l'ensemble des requêtes le jour de l'épreuve pratique sur ordinateur.

Commande SQL	Rôle et Précautions Clés	Exemple type de l'examen
INSERT INTO	Insère un nouvel uplet. Toujours cibler les colonnes pour la portabilité.	INSERT INTO ELEVE (Id, Nom) VALUES (1, 'Fedi');
UPDATE	Modifie un ou plusieurs attributs. Attention à ne pas oublier le WHERE.	UPDATE ELEVE SET Note = 15 WHERE Id = 1;
DELETE FROM	Supprime des enregistrements. Attention à ne pas oublier le WHERE.	DELETE FROM ELEVE WHERE Note < 10;
SELECT ... FROM	Extrait des colonnes d'une table (Projection algébrique).	SELECT Nom, Prenom FROM ELEVE;
WHERE (Restriction)	Filtre les lignes selon des conditions logiques complexes.	WHERE Moyenne >= 10 AND Ville = 'Tunis';
WHERE (Jointure)	Relie deux tables via leurs attributs clés d'association.	WHERE ELEVE.IdClasse = CLASSE.IdClasse;
GROUP BY	Regroupe les lignes pour appliquer des calculs statistiques par bloc.	SELECT IdClasse, AVG(Note) GROUP BY IdClasse;
HAVING	Filtre les groupes calculés (Conditions sur fonctions d'agrégation).	HAVING COUNT(*) > 25;
ORDER BY	Trie le résultat final par ordre croissant (ASC) ou décroissant (DESC).	ORDER BY Note DESC, Nom ASC;

3.3 Fiche Mémo Visuelle : Les 5 pièges classiques du SQL au Bac

1. **Le piège de l'omission de la jointure** : Écrire FROM Table1, Table2 sans ajouter WHERE Table1.PK = Table2.FK. *Sanction : Produit cartésien immédiat et note de zéro à la question.*
2. **La confusion WHERE et HAVING** : Écrire WHERE COUNT(*) > 10. *Règle d'or : Les fonctions d'agrégation ne s'évaluent jamais dans le WHERE. Utilisez GROUP BY suivi de HAVING.*
3. **Le désastre de l'UPDATE ou du DELETE sans WHERE** : Exécuter une modification de masse par inadvertance. *Vérifiez toujours votre filtre WHERE avant de valider votre code.*
4. **La syntaxe incorrecte du LIKE** : Écrire LIKE 'A*' à la place de LIKE 'A%' (influence d'Access) ou oublier d'entourer le filtre de guillemets simples.
5. **L'ambiguïté des noms de colonnes** : Sélectionner IdClasse sans préciser la table d'origine dans une jointure. *Utilisez toujours les préfixes (ex: E.IdClasse) pour éviter le message d'erreur « Ambiguous column name ».*